

RAG Retrieval-Augmented Generation_t

1. What is RAG?

RAG stands for Retrieval-Augmented Generation. It is a technique where you connect an LLM to an external knowledge base so it can look things up before it answers. Instead of relying only on what it learned during training, the model retrieves relevant documents in real time and uses them as context when generating a response.

This is important because LLMs have a knowledge cutoff — they do not know about recent events, your private company data, or anything that was not in their training set. RAG fixes that by letting the model fetch fresh, relevant information every time someone asks a question.

One-liner for interviews: RAG = give the LLM a search engine before it answers.

Why RAG instead of just fine-tuning?

Aspect	RAG	Fine-tuning
Knowledge updates	Easy — just update the docs	Hard — retrain the model
Cost	Cheaper to run	Expensive to train
Accuracy on specific data	Very good	Good but can overfit
Transparency	You can show sources	Black box
Best for	Dynamic, changing knowledge	Style, tone, domain language

2. RAG Pipeline — End to End

The RAG pipeline has two main phases: Indexing (offline, done once or periodically) and Querying (online, happens every time a user asks something).

Phase 1 — Indexing (Build the Knowledge Base)

This is the setup phase. You process your documents and store them so they can be searched later.

- **Step 1 — Load documents:** Pull in your raw data. This can be PDFs, Word docs, web pages, databases, CSVs — anything.
- **Step 2 — Clean and pre-process:** Remove noise, headers, footers, duplicate content. Normalize text formatting.
- **Step 3 — Chunk:** Split documents into smaller pieces (more on this in Section 3). Each chunk becomes a searchable unit.
- **Step 4 — Embed:** Convert each chunk into a vector (a list of numbers) using an embedding model like text-embedding-3 or sentence-transformers.
- **Step 5 — Store in vector database:** Save these vectors along with the original chunk text in a vector store (Pinecone, Weaviate, Chroma, pgvector, etc.)

Phase 2 — Querying (Answer a User's Question)

This happens live every time a user sends a message.

- **Step 1 — Receive user query:** The user types a question.
- **Step 2 — Embed the query:** Run the same embedding model on the question to get a query vector.
- **Step 3 — Retrieve relevant chunks:** Search the vector database for the top-k chunks most similar to the query vector (using cosine similarity or dot product).
- **Step 4 — Re-rank (optional):** recommended): Use a cross-encoder model to re-score the retrieved chunks for relevance. Keeps only the best ones.
- **Step 5 — Build the prompt:** Combine the user query + retrieved chunks into a prompt: 'Based on the following context, answer the question...'
- **Step 6 — Generate the answer:** Pass the prompt to the LLM. It generates a response grounded in the retrieved context.
- **Step 7 — Return response with sources:** Send the answer back to the user, ideally with citations to the source chunks.

Key thing to remember: The embedding model used at indexing time **MUST** be the same one used at query time. Mixing models breaks retrieval.

Hybrid Retrieval — Best of Both Worlds

Modern RAG systems often combine two types of search:

- **Dense retrieval:** Uses embedding vectors. Good at understanding meaning and context. (e.g. finds 'car' when you search 'vehicle')
- **Sparse retrieval (BM25):** Traditional keyword matching. Great at exact matches and rare terms.

Combining both (called Hybrid Search) and then applying re-ranking gives you the best retrieval accuracy.

3. Chunking Strategies

Chunking is how you split your documents before storing them. This step matters a lot. If your chunks are too big, the LLM gets irrelevant text. If they are too small, you lose important context.

Strategy 1 — Fixed Size Chunking

Split the document into chunks of a fixed number of tokens or characters. Simple and fast.

- Example: every 200 tokens, new chunk
- Use overlap: let consecutive chunks share some tokens (e.g. 20 tokens overlap) so context does not get cut off at boundaries
- Pro: fast, easy to implement, predictable
- Con: ignores sentence boundaries, can split ideas in half
- Best for: quick prototypes, long uniform documents

Strategy 2 — Sliding Window Chunking

Similar to fixed size but you slide a window across the text with overlap, so each chunk shares content with the previous one.

- Example: 200 token window, move 100 tokens at a time (50% overlap)
- Reduces the risk of a key sentence falling at the boundary and being missed
- Pro: better context continuity
- Con: stores more chunks, takes more storage and retrieval time
- Best for: dense technical documents where context spans multiple sentences

Strategy 3 — Recursive / Structure-Aware Chunking

Split on natural language boundaries first — paragraphs, then sentences, then words. LangChain's RecursiveCharacterTextSplitter does this.

- It tries to split on paragraph breaks first. If a chunk is still too big, it splits on sentences. Then words.
- Preserves semantic structure much better than fixed splitting
- Pro: respects natural writing structure, great starting point for most projects
- Con: chunk sizes can be uneven
- Best for: general use, most document types. Recommended default.

Strategy 4 — Semantic Chunking

Uses an embedding model to group sentences that are semantically similar together into one chunk. Splits happen at points where meaning changes.

- Embed every sentence, measure cosine similarity between consecutive sentences
- When similarity drops below a threshold, that is a chunk boundary
- Pro: chunks are truly coherent by meaning, not just by length
- Con: expensive — you need to embed every sentence just to do the chunking
- Research shows 70% better retrieval in some cases, but a NAACL 2025 paper found 200-token fixed chunks match or beat semantic chunking on many tasks
- Best for: high-stakes production systems where retrieval quality is critical

Strategy 5 — Document-Based / Hierarchical Chunking

Respect the document's own structure — headers, sections, subsections, tables.

- A PDF's Section 2.3 stays as one chunk. A table stays together.
- Often combined with metadata tagging (store the section title, page number, doc name with each chunk)
- Pro: chunks are logically meaningful units
- Con: requires document parsing logic

- Best for: structured documents like reports, legal contracts, manuals

Quick Reference — When to Use What

Strategy	Speed	Quality	Best For
Fixed Size	Very Fast	Basic	Prototypes, uniform text
Sliding Window	Fast	Good	Dense technical docs
Recursive (LangChain default)	Fast	Good	General purpose — start here
Semantic	Slow	Best	Production, high-stakes systems
Document-Based	Medium	Very Good	Structured PDFs, legal, manuals

Golden rule: Start with Recursive chunking + overlap. Only switch to semantic chunking if retrieval quality is still bad after tuning.

4. In-Memory vs Persistent Memory in RAG

When people say 'memory' in the context of RAG and AI agents, they mean how the system stores and accesses information. There are two main types.

In-Memory (Short-Term Memory)

Data lives in RAM. It is fast but disappears when the session ends or the server restarts.

- Used for: conversation history within a single session, temporary caches, fast prototyping
- Technologies: Python dictionaries, FAISS (in-memory mode), Chroma (in-memory mode), LangChain's ConversationBufferMemory
- Speed: very fast — no disk I/O, no network calls. Sub-millisecond retrieval.
- Limitation: nothing survives a restart. Not suitable for user data across sessions.

When to use in-memory:

- Single-session chatbots where you need to remember the conversation so far
- Development and testing — no setup needed
- Situations where you need ultra-low latency and the data is small enough to fit in RAM

Persistent Memory (Long-Term Memory)

Data is stored on disk or in a cloud database. It survives restarts and can grow very large.

- Used for: user profiles, conversation history across sessions, company knowledge bases, multi-user systems
- Technologies: Pinecone, Weaviate, Qdrant, Chroma (persistent mode), pgvector (PostgreSQL extension), Milvus, Redis Vector
- Speed: 10-100ms query latency (from Pinecone benchmarks). Slower than in-memory but fine for most use cases.
- Benefit: data persists forever, can handle millions of vectors, supports multi-user access

When to use persistent:

- Production systems where data cannot be lost
- Any multi-session use case (user comes back tomorrow and you remember their history)
- Large knowledge bases that cannot fit in RAM
- Multi-tenant applications where different users have different data

Memory in Agentic RAG Systems

Modern AI agents use multiple memory layers at once:

Memory Type	What it stores	Technology	Scope
Working memory	Current conversation context	In-memory (token window)	Current session only
Episodic memory	Past conversations and interactions	Persistent vector DB	Per user, long-term
Semantic memory	Facts, documents, knowledge base	Persistent vector DB	Shared across users
Procedural memory	How to do tasks (tools, skills)	Code / tool definitions	System-wide

Interview tip: Most real systems use BOTH. In-memory for the current session context, persistent for the knowledge base and user history.

Popular Vector Databases at a Glance

Database	Type	Best For
FAISS	In-memory / local	Fast local dev, research
Chroma	Both	Easy to start, good for prototypes
Pinecone	Managed cloud	Production, fully managed, no infra
Weaviate	Self-hosted / cloud	Complex filtering + vectors
pgvector	PostgreSQL extension	If you already use Postgres
Milvus / Zilliz	Self-hosted / cloud	High scale, billions of vectors

5. Common RAG Issues and How to Fix Them

Most RAG problems fall into one of three categories: retrieval is bad, generation is bad, or the data itself is bad. Here is a breakdown of the most common ones and exactly what to do about them.

Issue 1 — Hallucination (Model Makes Things Up)

The model generates information that is not in the retrieved context and not factually correct.

Why it happens:

- Retrieval failed — irrelevant chunks were fetched so the model had nothing useful to work with
- The model relied on its training knowledge instead of the context
- Prompt did not clearly tell the model to stick to the context

How to fix it:

- Improve retrieval quality first (better embeddings, better chunking)
- Add explicit instructions in your prompt: 'Only answer using the provided context. If the answer is not in the context, say I do not know.'
- Use faithfulness evaluation — score how grounded each answer is to its source chunks
- Add a post-generation check: use a second LLM call to verify the answer against the retrieved docs

Issue 2 — Retrieved Chunks Are Not Relevant

The system retrieves chunks but they are about the wrong topic or too generic to be useful.

Why it happens:

- Chunks are too big and contain many mixed topics
- The embedding model is not well-suited to your domain
- No re-ranking step to filter out weak matches

How to fix it:

- Reduce chunk size and add more overlap
- Try a domain-specific embedding model (e.g. BioBERT for medical, legal-bert for legal)
- Add a re-ranker (cross-encoder model like ms-marco-MiniLM) after retrieval to filter by true relevance
- Use hybrid search: combine dense embeddings with BM25 keyword search
- Add metadata filters: only search within the relevant document type, date range, or category

Issue 3 — Context Window Gets Overloaded

You retrieve too many chunks and they do not all fit in the LLM's context window, or the model gets confused by too much information.

How to fix it:

- Reduce top-k (retrieve fewer chunks, maybe 3-5 instead of 10)
- Use re-ranking to keep only the top 2-3 most relevant chunks
- Use toolsets or selective retrieval: only pull chunks if confidence score is above a threshold
- Use a model with a larger context window if you genuinely need more context

Issue 4 — Answer Ignores Retrieved Context (Lost in the Middle)

Research shows LLMs tend to ignore information in the middle of long prompts. They pay most attention to the beginning and end.

How to fix it:

- Put the most relevant chunk first or last in the prompt, not in the middle
- Reduce the number of chunks so there is less middle content
- Explicitly structure the prompt to reference each chunk: 'Use Source 1... Use Source 2...'

Issue 5 — Stale or Outdated Information

The knowledge base has not been updated so the model retrieves old information and gives wrong answers.

How to fix it:

- Set up automated re-indexing pipelines that update the vector store when documents change
- Add metadata (last_updated date) to every chunk and filter out chunks older than a threshold
- For live data, consider real-time connectors instead of static indexed documents

Issue 6 — Slow Query Response Time

The retrieval step takes too long, making the whole system feel slow.

How to fix it:

- Use ANN (Approximate Nearest Neighbor) search instead of exact search — faster with minimal accuracy loss (HNSW algorithm is the most common)
- Cache frequent query embeddings so you do not re-compute them every time
- Use a smaller, faster embedding model for retrieval (save the big one for re-ranking)
- Reduce vector dimensionality with PCA or use quantized vectors

Issue 7 — Poor Performance on Specific Document Types

RAG works fine on plain text but struggles with tables, charts, or scanned PDFs.

How to fix it:

- Use document parsers like Unstructured.io or Azure Document Intelligence for complex layouts
- Keep tables together as a single chunk rather than splitting across rows
- For scanned PDFs: use OCR first (Tesseract, AWS Textract, Google Document AI)
- For charts: use multimodal models that can read images

6. Issues Quick Reference

Issue	Root Cause	Fix
Hallucination	Bad retrieval or ignored context	Better chunks, stricter prompt, faithfulness check
Irrelevant chunks	Wrong embeddings, big chunks	Re-rank, smaller chunks, hybrid search
Context overload	Too many chunks	Lower top-k, re-rank to top 3
Lost in the middle	LLM ignores middle content	Put best chunk first, fewer chunks
Stale info	Outdated knowledge base	Auto re-indexing, date metadata filter

Issue	Root Cause	Fix
Slow response	Slow vector search	ANN/HNSW index, caching, smaller model
Complex doc formats	Tables, PDFs, charts	Document parsers, OCR, keep tables intact

Overall RAG debugging order: Always fix retrieval first (chunks + embeddings + re-ranking), then fix the prompt, then fix the model. Most problems are retrieval problems.